

HANDS-ON 2: SDLC vs TDLC - V-Model & Agile QA Task 1: V-Model Mapping 1.1 Complete V-Model Diagram

```

text +=====+ | ACCEPTANCE
TESTING (UAT) | | ----- | | * Verifies requirements | | * End-user validation |
| * Business sign-off | +=====+ | |
+=====+ | SYSTEM TESTING (ST) | | -
----- | | * Verifies system design | | * End-to-end functionality | | *
Performance, security, reliability |
+=====+ | |
+=====+ | INTEGRATION TESTING
(IT) | | ----- | | * Verifies architecture design | | * Interface testing | | *
Module integration | +=====+ | |
+=====+ | UNIT TESTING (UT) | | ----
----- | | * Verifies module design | | * Code-level testing | | * Individual
functions | +=====+ | |
+=====+ | CODING | | -----
----- | | * Implementation | | * Code review | | * Development |
+=====+

```

LEFT SIDE (Development) | RIGHT SIDE (Testing)

```

=====|=====
===== Requirements Phase | Acceptance Testing (UAT) System Design Phase | System Testing (ST)
Architecture Design Phase | Integration Testing (IT) Module Design Phase | Unit Testing (UT) Coding Phase |
(No corresponding testing phase) 1.2 SDLC ↔ TDLC Phase Mapping text +-----+-----
-----+-----+ | SDLC Phase | TDLC Phase | Test Artifact Produced | +-----
-----+-----+ | Requirements Phase | Acceptance
Testing | Acceptance Test Plan, UAT Test Cases, | | | Business Scenarios | +-----+-----
-----+-----+ | System Design | System Testing | System Test Plan, Performance
Test | | | Plan, Security Test Cases | +-----+-----+-----
-----+ | Architecture Design | Integration Testing | Integration Test Cases, API Test | | | Cases, Interface Test
Cases | +-----+-----+-----+ | Module Design |
Unit Testing | Unit Test Cases, Code Coverage | | | Targets, Test Stubs | +-----+-----
-----+-----+ | Coding Phase | Unit Testing | Test Code, Test Execution Reports, | |
| Execution | Code Coverage Reports | +-----+-----+-----
-----+ 1.3 Entry and Exit Criteria for Each Testing Level Unit Testing

```

text Entry Criteria:

- Code is complete and compiles without errors
- Unit test environment is set up
- Code review is completed
- Test data is available

Exit Criteria:

- 90%+ code coverage achieved
- All unit tests passed
- No Critical/High severity defects open
- Code is merged to main branch Integration Testing

text Entry Criteria:

- Unit testing is 100% complete
- Test environment with integrated modules is ready
- Test data is available
- Integration test plan is approved

Exit Criteria:

- All integration test cases executed
- All interfaces validated
- No Critical/High severity defects open
- Performance benchmarks met System Testing

text Entry Criteria:

- Integration testing is 100% complete
- Complete system deployed to test environment
- Performance test environment is ready
- System test plan is approved

Exit Criteria:

- All functional test cases executed
- All non-functional requirements validated
- Performance, security, and reliability benchmarks met
- No Critical defects open User Acceptance Testing (UAT)

text Entry Criteria:

- System testing is 100% complete
- UAT environment is ready
- Business users are available
- UAT test plan is approved

Exit Criteria:

- All UAT test cases executed
- All business requirements validated
- Business sign-off received
- Production deployment approved 1.4 Early QA Engagement Points Engagement Point 1: Requirements Review Phase

text QA reviews requirements before development starts Identifies ambiguities, missing requirements, and testability issues Example: QA asks "What happens when a course code is duplicated?" during requirements review Benefit: Prevents defects before code is written (saves 10x cost) Engagement Point 2: Architecture Design Phase

text QA reviews design specifications Identifies potential testability issues Prepares test strategy and automation plan Example: QA recommends adding unique IDs to all UI elements for automation Benefit: Ensures system is designed for testability Task 2: Agile QA and Shift-Left Testing 2.1 Waterfall Problems text

Problem 1: Late Defect Discovery Impact on Course Management API Project: Bugs found at the end of the cycle (during system testing) are expensive to fix. A course creation bug found late requires rework of design, code, and tests.

Problem 2: Limited Test Time Impact on Course Management API Project: Testing is compressed at the end because development took longer than expected. QA has only 2 days to test all course management features, leading to incomplete testing.

Problem 3: Communication Gaps Impact on Course Management API Project: QA is not involved in requirements and design. Developers implement course API based on their interpretation, leading to mismatches between expected and actual behavior.

2.2 QA Role in Agile Ceremonies

Sprint Planning:

- Define and review acceptance criteria for each user story
- Estimate testing effort for each story
- Identify test dependencies
- Plan test automation scope

Daily Standup:

- Report testing progress
- Raise blocking issues (e.g., test environment down)
- Coordinate with developers on bug fixes
- Share test results

Sprint Review:

- Demo testing results to stakeholders
- Demonstrate automated tests
- Show test coverage metrics
- Validate that acceptance criteria are met

Sprint Retrospective:

- Identify testing process improvements
 - Discuss flaky tests and their causes
 - Share lessons learned
 - Plan improvements for next sprint
- ## 2.3 Shift-Left Practices
- ### Practice 1: Reviewing Requirements for Testability

text Application to Course Management API: During requirements review, QA asks "How should the system handle duplicate course codes?" and "What happens when 100 admins create courses simultaneously?" Result: Requirements become clearer, preventing ambiguous implementations.

Practice 2: Writing Test Cases Before Code (TDD/BDD)

text Application to Course Management API: Write Gherkin scenarios for course creation before developers write code: Given I am an admin When I create a course with valid data Then the course should be created Result: Tests define expected behavior first, guiding development.

Practice 3: Static Code Analysis

text Application to Course Management API: Use tools like SonarQube to analyze the API code for security vulnerabilities, code smells, and complexity issues before integration. Result: Identifies coding issues before testing begins.

Practice 4: API Contract Testing

text Application to Course Management API: Test API contracts using tools like Pact or Postman before frontend and backend are fully integrated. Result: Catches breaking changes early, preventing integration failures. 2.4 Acceptance Criteria in Gherkin Format User Story: As a college admin, I want to create a new course, so that students can enroll in it.

Scenario 1: Happy Path (Valid Data)

text Feature: Course Creation As a college admin I want to create a new course So that students can enroll in it

Scenario: Admin creates a course with valid data Given I am logged in as an admin And I am on the "Course Management" page When I click on "Add New Course" And I enter course name "Data Structures" And I enter course code "CS201" And I enter credits "4" And I select instructor "Dr. Smith" And I click "Save Course" Then I should see "Course created successfully" And the new course should appear in the course list And the course status should be "Active" Scenario 2: Duplicate Course Code

text Scenario: Admin attempts to create a course with duplicate code Given I am logged in as an admin And I am on the "Course Management" page And a course with code "CS201" already exists When I click on "Add New Course" And I enter course name "Advanced Programming" And I enter course code "CS201" And I enter credits "4" And I select instructor "Dr. Jones" And I click "Save Course" Then I should see "Course code already exists" And the course should NOT be created And I should remain on the course creation page Scenario 3: Missing Required Fields

text Scenario: Admin attempts to create a course with missing required fields Given I am logged in as an admin And I am on the "Course Management" page When I click on "Add New Course" And I enter course name "Data Structures" And I leave course code blank And I enter credits "4" And I select instructor "Dr. Smith" And I click "Save Course" Then I should see "Course code is required" And the course should NOT be created And I should remain on the course creation page And all previously entered data should still be visible Key Takeaways text

1. V-Model: Left side = Development, Right side = Testing
2. Entry Criteria: Pre-conditions for testing phase to begin
3. Exit Criteria: Conditions to complete testing phase
4. Shift-Left: Move testing activities earlier in the cycle
5. Agile QA: QA is a full team member from the start
6. TDD/BDD: Write tests before code
7. Gherkin: Given-When-Then format for acceptance criteria
8. Early Engagement: QA involvement in requirements and design phases